

Project Proposal: A Guided Tool for Testing Trading Strategies

Chinmay Karur, ckarur@calpoly.edu

1. Problem Statement

Backtesting tools let anyone simulate a trading strategy over years of price history in seconds, but for a regular investor the result is basically a black box. They default to zero trading costs, so the return leaves out the fees that decide whether a strategy makes money at all. In my own testing, turning costs on changed a simulated return from 52% to 5%. They also let you try hundreds of parameter combinations and show you the best one, which looks good even if the strategy is worthless, and they measure it on the same data used to pick the settings. When I split the data instead, the settings that scored best on the first half (+81%) returned +32% on the second half, while the worst ones (−18%) returned +21%.

To combat this, I want to build a web app that does the testing properly and reports one verdict, either Robust, Weak, or Overfit, instead of a return number. A strategy that passes can then be started in paper trading, which runs on live prices with fake money.

2. Goals

- Build a backtest engine in Python with commission, slippage, stop losses and position sizing.
- Check it against TradingView by running the same strategies through both.
- Turn the statistical checks into a component that outputs one verdict.
- Build a web app that goes from picking a market to running paper trading, first on crypto and then US stocks.
- Add a classifier that filters strategies by market condition.

3. Design

1. Ask the user for a market, a holding time and how much risk they will take.
2. Pick a strategy from the library and generate a range of settings, or let the user describe their own strategy in text.
3. Download and cache the price history, then split it: an earlier part to pick settings, a later part to score them.
4. Simulate every combination over the same dates, charging commission and slippage on each trade.
5. Rank on the later part only, adjust for how many were tried, and report a verdict.

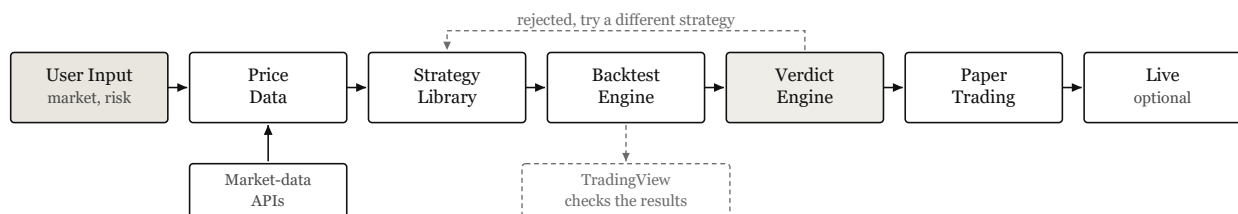


Figure 1: The web app's backend pipeline.

Inputs: the user's market, holding time and risk, a strategy from the library or written by the user, and price history with a cost model.

Outputs: a ranked short list of settings, each with a verdict and a one-sentence reason, plus a paper-trading session for the chosen one.

4. Prior Work

Brock et al. [1] is the standard reference for testing simple trading rules. Sullivan et al. [2] retested it against a far larger set of rules: the performance survived a century of data, but vanished over the following decade, which is the argument for scoring on a period the settings were not chosen on. Bailey et al. [3] call the general problem backtest overfitting, Harvey and Liu [4] give a way to discount a result by how many settings were tried, and Arnott et al. [5] collect the fixes into a checklist done by hand. TradingView, Backtrader and QuantConnect leave that checklist to the user; my project runs it automatically.

5. Technical Approach

- Python backend, vectorized simulation over cached data, results in SQLite so a long run can be stopped and resumed. Next.js frontend on a REST API, with a cloud-hosted model called through OpenRouter to turn a strategy written in plain text into settings the engine can run.
- Trading execution runs in its own process, so a bug in the research code cannot place an order. Automated tests cover the engine.

6. Timeline

Weeks	Focus	Deliverable
1–4	Price data, backtest engine, verdict engine.	Reusable local datasets, backtest engine, verdict engine.
5–8	Start the web app and the UI that takes user input, hooked up to the backtest and verdict engines.	Working web app end to end.
9–12	Live paper trading and deployment.	Web app pipeline extended with paper trading.
13–14	Testing and final touches.	Final report and demo.

7. References

1. Brock, W., Lakonishok, J., & LeBaron, B. (1992). Simple Technical Trading Rules and the Stochastic Properties of Stock Returns. *Journal of Finance*, 47(5), 1731–1764. [PDF: finance.martinsewell.com](https://finance.martinsewell.com) (scan)
2. Sullivan, R., Timmermann, A., & White, H. (1999). Data-Snooping, Technical Trading Rule Performance, and the Bootstrap. *Journal of Finance*, 54(5), 1647–1691. [PDF: eprints.lse.ac.uk](https://eprints.lse.ac.uk) (working paper)
3. Bailey, D. H., Borwein, J. M., López de Prado, M., & Zhu, Q. J. (2014). Pseudo-Mathematics and Financial Charlatanism. *Notices of the AMS*, 61(5), 458–471. [PDF: ams.org](https://ams.org)
4. Harvey, C. R., & Liu, Y. (2015). Backtesting. *Journal of Portfolio Management*, 42(1), 13–28. [PDF: people.duke.edu](https://people.duke.edu)
5. Arnott, R. D., Harvey, C. R., & Markowitz, H. (2019). A Backtesting Protocol in the Era of Machine Learning. *Journal of Financial Data Science*, 1(1), 64–74. [PDF: people.duke.edu](https://people.duke.edu)
6. Docs: docs.ccxt.com, docs.alpaca.markets, tradingview.com/pine-script-reference/v6, openrouter.ai/docs